Predicting Pokémon Battles by Rebuilding What the Log Hides

Matteo Angeloro Lorenzo Amoruso Carmine [TODO: surname]
Lorenzo Valenti
Sapienza University of Rome
[TODO: institutional emails]

Abstract

We predict whether Player 1 wins a Gen 2 OU Pokémon battle from its first 30 recorded turns. The key observation is that the battle log hides most of Player 2's knockouts: a fainted Player 2 Pokémon looks like an ordinary switch. Using Player 1's always-visible knockouts as a control group, a classifier estimates which Player 2 switches were knockouts, and the corrected game state feeds a LightGBM model together with an out-of-fold encoding of Player 1's team history. The model reaches 0.699 accuracy in repeated cross-validation, 0.700 on the public leaderboard and 0.683 on the private one.

1. Introduction

We are given 10,000 labelled and 5,000 test Gen 2 OU battles. From Player 1's team, Player 2's lead and the first 30 recorded turns we must say whether Player 1 wins. Battles last 30 to 80 turns, so the log stops while the outcome is still open.

What does the log really show? Two facts emerged. Player 2's `fnt` state appears almost only in double knockouts: about one Player 2 knockout per battle looks like an ordinary switch. And the same Player 1 team recurs across battles, with a win rate that predicts the outcome, while Player 2's does not. How can we turn these facts into features?

2. Evaluated Approaches

Feature engineering. We started from the state at turn 30 taken at face value: Pokémon alive and their HP, statuses, the active Pokémon, boosts and Spikes. This scored 0.675. Adding Player 1's species and items and the difference of move counts between the players gave a baseline of 0.685.

This state still counts Player 2's hidden knockouts as alive. How can we find them? Player 1's knockouts are always visible, so every Player 1 Pokémon that leaves and never comes back (a "vanishing") is a real switch. If both players switch alike, the extra vanishings of Player 2 are the hidden knockouts. A LightGBM classifier separates Player 2's vanishings from Player 1's using only the event (HP before leaving, whether the replacement entered unhurt, move powers). With q its output, the share of real switches among Player 2's events is $(1-q)/q$, so

$$P(\mathrm{KO} \mid x) = 1 - c\,\frac{1-q}{q},$$

where c makes the total match the observed excess. No labels are used, and Player 2 now loses as many Pokémon as Player 1 (1.61 vs. 1.69 per battle). We recompute the state weighting each Pokémon by its probability of being alive.

What about Player 1's identity? We add the smoothed win rate of Player 1's exact team, as a target encoding computed out-of-fold inside each training fold.

Models. Our learner is LightGBM with shallow trees; logistic regression was 0.5 points worse, and XGBoost, CatBoost and blends stayed within ± 0.003.

Validation. With differences this small, how do we know a block really helps? Every configuration uses the same stratified 5-fold CV repeated three times, and we compare paired fold differences with the Nadeau–Bengio corrected standard error [1], keeping a block only if it helps. This dropped Player 2's estimated bench, extra state features and a damage formula.

3. Results

Table 1. CV accuracy and paired gain over the baseline.

Configuration	CV acc.	Gain
State at face value	0.675	-0.010
Baseline (+ species, items, moves)	0.685	–
Final without hidden knockouts	0.687	$+0.002$
Final without P1 history	0.697	$+0.012$
Final model	**0.699**	$\mathbf{+0.014 \pm 0.004}$

Which block matters? Table 1 answers: almost all of the gain comes from the hidden-knockout correction ($+0.012 \pm 0.003$); the history adds $+0.002 \pm 0.004$ once species are present. It scored 0.700 on the public and 0.683 on the private leaderboard. The private score is 1.5 points below CV, about two standard errors of a 3,500-battle split. Code at https://github.com/teoang/fds-pokemon-challenge.

References

[1] Claude Nadeau and Yoshua Bengio. Inference for the generalization error. *Machine Learning*, 52(3):239–281, 2003. 1